CORTE 3 – ENTREGA 1

Desarrollo de ejercicios de LeetCode y Exercism utilizando C++

Integrantes

- Camila Garces Castro
- Andres Felipe Pinilla

Repositorio GitHub

https://github.com/Cami-g06/corte3_ent1.git

INTRODUCCIÓN

Durante el aprendizaje de programación es necesario desarrollar habilidades relacionadas con el análisis de problemas, la selección adecuada de estructuras de datos y el diseño de algoritmos eficientes. En muchos casos, un programa funcional no es suficiente; también es importante que la solución sea capaz de responder de forma rápida cuando la cantidad de datos aumenta.

Las plataformas LeetCode y Exercism son herramientas ampliamente utilizadas para fortalecer estas competencias. A través de ejercicios prácticos permiten enfrentarse a situaciones similares a las encontradas en el desarrollo de software profesional. Los problemas suelen requerir el uso de estructuras de datos avanzadas, algoritmos optimizados y conceptos de programación orientada a objetos.

En esta entrega se desarrollaron tres ejercicios de la plataforma LeetCode y dos ejercicios de la plataforma Exercism. Cada ejercicio fue analizado individualmente con el propósito de comprender sus restricciones, seleccionar las estructuras de datos apropiadas y construir una solución eficiente utilizando el lenguaje C++17.

Además de implementar las soluciones, se realizó un estudio de complejidad temporal y espacial para determinar la eficiencia de cada algoritmo. También se documentaron las dificultades encontradas durante el desarrollo y las decisiones tomadas para resolver cada problema.

Los ejercicios seleccionados abarcan diferentes áreas de la programación:

- Gestión de información en tiempo real.
- Almacenamiento eficiente de datos.
- Cálculo de estadísticas acumuladas.

- Algoritmos voraces.
- Manejo de fechas.
- Manipulación de tiempo.
- Programación orientada a objetos.
- Uso de la Standard Template Library (STL).

El proyecto final fue organizado mediante CMake, permitiendo compilar todas las soluciones desde una única configuración y facilitando la administración del código fuente.

OBJETIVOS

Objetivo general

Desarrollar soluciones eficientes para los ejercicios propuestos aplicando estructuras de datos, programación orientada a objetos y análisis de complejidad mediante el uso del lenguaje C++.

Objetivos específicos

- Comprender los requerimientos de cada problema.
- Analizar las restricciones de entrada y salida.
- Diseñar algoritmos adecuados para cada situación.
- Utilizar estructuras de datos eficientes.
- Implementar soluciones funcionales en C++17.
- Evaluar la complejidad temporal y espacial de cada algoritmo.
- Organizar los proyectos utilizando CMake.
- Documentar el proceso de desarrollo.

METODOLOGÍA DE DESARROLLO

Para el desarrollo de la actividad se siguió una metodología compuesta por varias etapas.

Inicialmente se realizó una lectura detallada de cada problema para comprender completamente los requisitos funcionales y las restricciones impuestas por la plataforma. Esta etapa resultó fundamental porque muchos ejercicios presentan condiciones específicas que deben respetarse estrictamente.

Una vez comprendido el problema se identificaron las posibles estructuras de datos que podrían utilizarse para resolverlo. En esta fase se compararon distintas alternativas evaluando tiempo de acceso, facilidad de actualización y consumo de memoria.

Posteriormente se desarrolló un diseño preliminar de la solución. Antes de escribir código se definieron las operaciones necesarias, la forma de almacenar la información y los algoritmos que permitirían responder a las consultas requeridas.

La implementación se realizó utilizando C++17 y las bibliotecas de la STL. Se procuró mantener un código modular, legible y fácil de mantener.

Finalmente se realizaron pruebas utilizando los casos proporcionados en los enunciados y se estudió la complejidad computacional de cada solución.

Las etapas ejecutadas fueron:

1. Lectura del problema.
2. Análisis de restricciones.
3. Selección de estructuras de datos.
4. Diseño de algoritmos.
5. Implementación.
6. Pruebas.
7. Análisis de complejidad.
8. Documentación.

LEETCODE 1396 – DESIGN UNDERGROUND SYSTEM

Descripción del problema

Este ejercicio plantea el desarrollo de un sistema de transporte subterráneo capaz de registrar los viajes realizados por sus usuarios. El sistema debe almacenar información sobre entradas y salidas de estaciones con el objetivo de calcular el tiempo promedio que los pasajeros tardan en desplazarse entre dos estaciones determinadas.

Las operaciones disponibles son:

checkIn()

checkOut()

getAverageTime()

Cada usuario posee un identificador único y puede encontrarse en una sola estación al mismo tiempo.

Análisis

La principal dificultad del problema consiste en administrar simultáneamente información temporal y estadísticas históricas.

Cuando un usuario realiza un check-in, el sistema necesita recordar la estación en la que inició el recorrido y la hora en la que ingresó. Posteriormente, cuando el mismo usuario realiza un check-out, es necesario recuperar esta información para calcular la duración del viaje.

Una solución simple podría almacenar todos los viajes realizados dentro de una colección y calcular el promedio recorriendo todos los registros cada vez que se solicite una consulta. Sin embargo, esta estrategia resulta poco eficiente, especialmente cuando la cantidad de viajes aumenta.

Por esta razón se optó por almacenar únicamente información acumulada de cada ruta. Para cada combinación origen-destino se registra:

- Tiempo total acumulado.
- Número total de viajes.

Gracias a esta estrategia el promedio puede obtenerse mediante una sola división.

Estructuras de datos utilizadas

Registro de usuarios activos

unordered_map<int, pair<string,int>>

Permite localizar instantáneamente la información de un pasajero.

Estadísticas de rutas

unordered_map<string,

pair<long long,int>>

Permite almacenar estadísticas acumuladas para cada trayecto.

Metodología

Cuando un pasajero ingresa al sistema se almacena:

- Identificador.
- Estación de origen.
- Hora de entrada.

Cuando sale:

1. Se recupera la información almacenada.
2. Se calcula la duración del viaje.
3. Se actualizan las estadísticas correspondientes.
4. Se elimina el registro temporal.

Finalmente el promedio se calcula utilizando:

promedio = tiempoTotal / totalViajes

Dificultades encontradas

La principal dificultad fue diseñar una estructura capaz de administrar pasajeros activos y estadísticas históricas de forma independiente pero consistente.

También fue necesario garantizar que las consultas de promedio se ejecutaran en tiempo constante independientemente de la cantidad de viajes acumulados.

Complejidad

Operación	Complejidad
checkIn	O(1)
checkOut	O(1)
getAverageTime	O(1)

Complejidad espacial:

O(n)

LEETCODE 2241 – DESIGN AN ATM MACHINE

Descripción del problema

El ejercicio consiste en implementar el comportamiento de un cajero automático que almacena billetes de las siguientes denominaciones:

20, 50, 100, 200 y 500

El sistema debe permitir realizar depósitos y retiros respetando una política específica: siempre intentar utilizar primero los billetes de mayor valor.

Análisis

Este problema resulta interesante porque obliga a seguir estrictamente una estrategia Greedy.

Normalmente, cuando se trabaja con problemas relacionados con cambio de monedas o billetes, se intenta encontrar cualquier combinación válida. Sin embargo, en este caso el comportamiento del cajero está previamente definido.

El sistema siempre debe intentar retirar dinero comenzando por los billetes de 500 dólares. Después utiliza los de 200, posteriormente los de 100, luego los de 50 y finalmente los de 20.

Esto provoca situaciones en las cuales una combinación válida existe, pero el retiro debe rechazarse porque el cajero no puede apartarse de la estrategia indicada.

Estructura utilizada

vector<long long>

Cada posición representa la cantidad de billetes disponibles de una determinada denominación.

Metodología

Para procesar un retiro se realiza el siguiente procedimiento:

1. Seleccionar la denominación más alta.
2. Tomar la máxima cantidad posible.
3. Reducir el valor restante.

4. Repetir el proceso para las siguientes denominaciones.
5. Verificar si el monto solicitado fue completado.

Si no es posible completar la cantidad exacta, la operación se rechaza.

Dificultades encontradas

Una de las principales dificultades fue comprender que el objetivo del problema no consiste en encontrar una combinación cualquiera, sino replicar exactamente la política definida por el cajero.

Otra dificultad consistió en mantener el estado interno del sistema sin modificaciones cuando una operación de retiro falla.

Complejidad

Operación	Complejidad
deposit	O(1)
withdraw	O(1)

Complejidad espacial:

O(1)

LEETCODE 3815 – DESIGN AUCTION SYSTEM

Descripción del problema

El ejercicio consiste en implementar un sistema de subastas que permita registrar ofertas realizadas por múltiples usuarios para diferentes artículos.

Cada oferta contiene:

userId

itemId

bidAmount

El sistema debe permitir agregar, modificar, eliminar y consultar ofertas.

Análisis

La principal dificultad consiste en determinar rápidamente cuál es la oferta ganadora de un artículo.

Una solución basada en vectores requeriría revisar todas las ofertas registradas cada vez que se solicite una consulta. Esta estrategia resulta poco eficiente cuando existe una gran cantidad de usuarios participando en la subasta.

Por esta razón se decidió utilizar una combinación de tablas hash y conjuntos ordenados.

Las tablas hash permiten localizar ofertas específicas de forma inmediata.

Los conjuntos ordenados mantienen las ofertas clasificadas automáticamente según su valor.

Estructuras utilizadas

Tabla hash

unordered_map

Utilizada para encontrar ofertas específicas.

Conjunto ordenado

set<pair<int,int>>

Utilizado para mantener las ofertas ordenadas según monto y usuario.

Manejo de empates

El problema establece que, cuando dos usuarios realizan ofertas con exactamente el mismo valor, debe considerarse ganador aquel cuyo identificador sea mayor.

Para resolver esta condición las ofertas fueron almacenadas de la siguiente forma:

(monto, usuario)

Gracias al comportamiento del conjunto ordenado, la última posición corresponde automáticamente a la oferta ganadora.

Metodología

Cuando un usuario crea una oferta:

1. Se almacena en la tabla hash.
2. Se registra en el conjunto ordenado.

Cuando actualiza una oferta:

1. Se elimina el valor anterior.
2. Se actualiza la tabla hash.
3. Se inserta nuevamente en el conjunto.

Esta estrategia mantiene ambas estructuras sincronizadas.

Dificultades encontradas

Las principales dificultades fueron:

- Manejar correctamente los empates.
- Sincronizar las estructuras utilizadas.
- Actualizar información sin generar inconsistencias.
- Mantener consultas rápidas incluso cuando el número de ofertas aumenta.

Complejidad

Operación	Complejidad
addBid	O(log n)
updateBid	O(log n)
removeBid	O(log n)
getHighestBidder	O(1)

Complejidad espacial:

O(n)

EXERCISM – MEETUP

Descripción del problema

El ejercicio solicita encontrar una fecha específica dentro de un calendario utilizando:

- Año.
- Mes.
- Día de la semana.
- Tipo de semana.

Las semanas posibles son:

first

second

third

fourth

last

teenth

Análisis

El problema requiere identificar una ocurrencia concreta de un día dentro de un mes.

Para resolverlo se decidió recorrer todos los días del mes y verificar cuáles coinciden con el día solicitado.

Posteriormente se selecciona la ocurrencia requerida según el tipo de semana indicado.

Aunque existen algoritmos capaces de calcular directamente la fecha buscada, la estrategia seleccionada resulta más sencilla de implementar y suficientemente eficiente debido al reducido tamaño del calendario mensual.

Caso especial: Teenth

La semana teenth corresponde a los días:

13

14

15

16

17

18

19

Dentro de este intervalo cada día de la semana aparece exactamente una vez.

Esta propiedad simplifica considerablemente la búsqueda.

Dificultades encontradas

- Manejo correcto de años bisiestos.
- Cálculo del día de la semana.
- Comprensión del concepto teenth.
- Conversión entre diferentes representaciones de fechas.

Complejidad

Tiempo:

O(31)

Memoria:

O(31)

EXERCISM – CLOCK

Descripción del problema

El objetivo de este ejercicio es implementar un reloj independiente de cualquier fecha.

El sistema debe permitir:

- Crear horas.
- Sumar minutos.

- Restar minutos.
- Comparar relojes.
- Mostrar la hora formateada.

Análisis

La estrategia utilizada consiste en representar la hora mediante la cantidad total de minutos transcurridos desde medianoche.

Por ejemplo:

00:00 = 0

01:00 = 60

12:00 = 720

23:59 = 1439

Esta representación simplifica todas las operaciones relacionadas con el tiempo.

Metodología

Todas las operaciones se realizan sobre una única variable entera.

Posteriormente se utiliza aritmética modular para mantener el resultado dentro de los límites válidos de un día.

((x % 1440) + 1440) % 1440

Esta expresión permite manejar correctamente valores negativos y valores superiores a 24 horas.

Ventajas de la solución

- Simplicidad de implementación.
- Menor cantidad de errores.
- Comparaciones sencillas.
- Normalización automática de la hora.

Dificultades encontradas

Una de las principales dificultades fue garantizar que todas las operaciones produjeran resultados válidos sin importar la cantidad de minutos sumados o restados.

También fue necesario gestionar correctamente las equivalencias entre diferentes representaciones de una misma hora.

Complejidad

Operación	Complejidad
Crear reloj	O(1)
Sumar minutos	O(1)
Comparar relojes	O(1)
Mostrar hora	O(1)

ORGANIZACIÓN DEL PROYECTO

El repositorio fue organizado mediante carpetas independientes para facilitar la localización y mantenimiento del código.

corte3_ent1/

├── README.md

├── CMakeLists.txt

├── leetcode

└── exercism

Todos los programas pueden compilarse desde una única configuración mediante CMake.

HERRAMIENTAS UTILIZADAS

- C++17
- Visual Studio Code

- Git
- GitHub

CMake

- STL

Bibliotecas utilizadas:

- unordered_map
- vector
- set
- string
- algorithm

CONCLUSIONES

El desarrollo de esta actividad permitió aplicar diversos conceptos fundamentales relacionados con programación, estructuras de datos y análisis algorítmico.

Los ejercicios de LeetCode demostraron la importancia de seleccionar estructuras de datos adecuadas para obtener soluciones eficientes. El uso de tablas hash permitió realizar búsquedas rápidas, mientras que los conjuntos ordenados facilitaron el mantenimiento automático de información clasificada.

Los ejercicios de Exercism reforzaron conceptos relacionados con programación orientada a objetos, representación de fechas y manipulación del tiempo mediante aritmética modular.

Además, la organización del proyecto mediante CMake permitió consolidar todas las soluciones dentro de una única estructura de compilación, facilitando el mantenimiento y la reutilización del código.

Finalmente, esta actividad permitió fortalecer habilidades de análisis, diseño de software y resolución de problemas, competencias fundamentales para el desarrollo de aplicaciones más complejas en cursos posteriores.